CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 13
NP-Completeness, Reductions & Exact Exponential Algorithms

How to prove that a problem is probably hard, and what to do on Monday morning when it is.

Saturday 19 December 2026 · 3 hours

Final report due Thu 24 Dec · PS4 due Sat 26 Dec

Motivating problem: an innocent feature request

A hospital scheduling tool already assigns nurses to shifts (a flow problem, L12). A new request:

> "Also make sure that no two nurses who are in conflict ever share a shift, and use as few distinct shift slots as possible."

The team spends three weeks looking for an efficient exact algorithm.

DISCUSS · 3 MIN

1. Which classic problem is hiding in "no two conflicting nurses in the same slot, minimise the number of slots"?

2. What would it take to *convince your manager* that nobody knows an efficient exact algorithm, and that none is likely to exist?

BLOCK A · 00:10–01:00

P, NP and reductions

Certificates · direction · what a proof requires

Decision problems, P and NP

- A **decision problem** = a set of yes-instances (encoded as strings). Optimisation problems have decision versions: "is there a colouring with $\leq k$ colours?"
- **P**: decidable in time polynomial in the input length.
- **NP**: yes-instances have **certificates** of polynomial length that a polynomial-time **verifier** accepts. No-instances have none.

$$x \in L \iff \exists y,\ |y| \leq p(|x|) :\ V(x, y) = 1$$

$\mathrm{P} \subseteq \mathrm{NP}$ (the verifier ignores y). Whether $\mathrm{P} = \mathrm{NP}$ is open.

Problem	Certificate	Verification
k-colouring	a colour per vertex	check each edge, count colours
Hamiltonian cycle	a vertex order	a permutation with consecutive edges
SAT	a truth assignment	evaluate the formula
Subset sum	a subset	add it up (in bits!)
"No Hamiltonian cycle"	?	not known to be in NP (it is in coNP)

Polynomial-time reductions and their direction

DEFINITION

$A \leq_p B$ if there is a polynomial-time computable f with $x \in A \iff f(x) \in B$ for every instance x.

Meaning: "A is no harder than B". An efficient algorithm for B gives one for A: compute $f(x)$, then run B's algorithm.

To show B is hard, reduce a known hard problem **to** B: $(\text{known hard}) \leq_p B$.

Transitivity: $A \leq_p B$ and $B \leq_p C$ imply $A \leq_p C$.

THE CLASSIC ERROR

"I reduced my problem B to SAT, so B is NP-hard." That shows only $B \in$ NP (no harder than SAT). The arrow points the wrong way.

MNEMONIC

The reduction transforms instances **of the known hard problem** into instances **of your problem**. You build gadgets *inside your problem*.

NP-hard, NP-complete, and what a proof requires

- **NP-hard:** every problem in NP reduces to it. It need not be in NP (e.g. optimisation versions, the halting problem).
- **NP-complete:** NP-hard *and* in NP.
- **Cook–Levin (1971):** SAT is NP-complete. Every NP verifier's computation can be encoded as a CNF formula.
- Consequence: if any NP-complete problem is in P, then P = NP.

RECIPE: PROVING X NP-COMPLETE

1. $X \in \mathrm{NP}$**:** describe a certificate and a polynomial verifier.
2. **Choose** a known NP-complete Y with a similar structure.
3. **Construct** f: instances of $Y \to$ instances of X. Argue it runs in polynomial time (output size!).
4. **(⇒)** y yes ⇒ $f(y)$ yes: map a solution forwards.
5. **(⇐)** $f(y)$ yes ⇒ y yes: map *any* solution of $f(y)$ backwards. This is often the hard part.

Exercise A: certificates and directions

1. Give a certificate and a verifier (with its running time) for: (a) "G has a clique of size $\geq k$"; (b) "the integer N is composite"; (c) "the knapsack instance has a solution with value $\geq V$".
2. True or false, with a one-line reason:
   a. If $A \leq_p B$ and A is NP-hard, then B is NP-hard.
   b. If $A \leq_p B$ and $B \in \mathrm{P}$, then $A \in \mathrm{P}$.
   c. If $A \leq_p B$ and B is NP-hard, then A is NP-hard.
   d. Every NP-hard problem is in NP.
   e. An $O(2^{\sqrt{n}})$ algorithm for an NP-complete problem would imply P = NP.

Pairs · 8 min

BLOCK B · 01:10–02:00

A chain of reductions

SAT → 3-SAT → Independent Set → Vertex Cover, Clique

SAT → 3-SAT

3-SAT: CNF where every clause has exactly 3 literals. In NP: the certificate is an assignment.

Reduction: transform each clause $C = (\ell_1 \vee \cdots \vee \ell_k)$ separately:

- $k = 1$: $(\ell_1 \vee y \vee z)(\ell_1 \vee y \vee \bar{z})(\ell_1 \vee \bar{y} \vee z)(\ell_1 \vee \bar{y} \vee \bar{z})$.
- $k = 2$: $(\ell_1 \vee \ell_2 \vee y)(\ell_1 \vee \ell_2 \vee \bar{y})$.
- $k = 3$: unchanged.
- $k > 3$: a chain with new variables $y_1 .. y_{k-3}$:

$$(\ell_1 \vee \ell_2 \vee y_1)(\bar{y}_1 \vee \ell_3 \vee y_2) \cdots (\bar{y}_{k-3} \vee \ell_{k-1} \vee \ell_k).$$

CORRECTNESS OF THE CHAIN ($k > 3$)

($\Rightarrow$) If some ℓ_i is true, set $y_j =$ true for $j < i - 1$ and false for $j \geq i - 1$. Every chain clause is then satisfied.

($\Leftarrow$) If all ℓ_i are false, the first clause forces y_1, which forces y_2, ..., y_{k-3}, and the last clause $(\bar{y}_{k-3} \vee \ell_{k-1} \vee \ell_k)$ is false. So some ℓ_i must be true.

Size: $O(k)$ new clauses per clause, so linear overall. ■

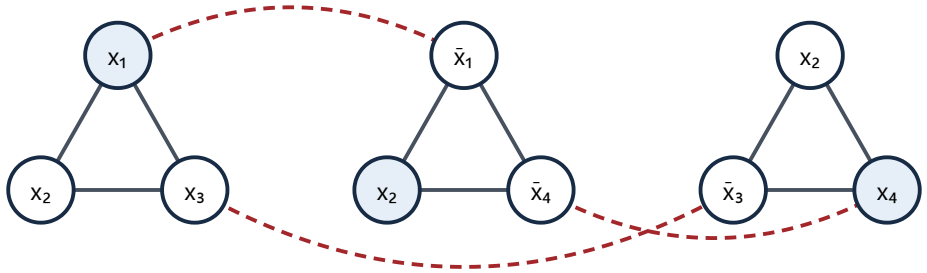
3-SAT → Independent Set

IS: does G have k pairwise non-adjacent vertices?

Construction from a 3-CNF φ with m clauses:

- a **triangle** for each clause (one vertex per literal occurrence);
- a **conflict edge** between every pair of complementary literals (x_i and $\bar{x}_i$) in different triangles;
- $k = m$.

Size: $3m$ vertices, $O(m^2)$ edges.



$(x_1 \vee x_2 \vee x_3)(\bar{x}_1 \vee x_2 \vee \bar{x}_4)(x_2 \vee \bar{x}_3 \vee x_4)$ · k = 3

shaded: $x_1 = x_2 = x_4$ = true → one vertex per triangle

PROOF

(⇒) Pick one true literal per clause: m vertices, one per triangle, never complementary, so independent. (⇐) An independent set of size m has exactly one vertex per triangle (triangles are cliques). No complementary pair is chosen, so setting the chosen literals to true is a consistent assignment (other variables arbitrary), and it satisfies every clause. ■

IS ↔ Vertex Cover ↔ Clique

COMPLEMENT LEMMA

S is an independent set of $G \Leftrightarrow V \setminus S$ is a vertex cover of G.

Proof: S is independent ⇔ no edge has both endpoints in S ⇔ every edge has an endpoint in $V \setminus S$.

So $(G, k) \in$ IS ⇔ $(G, n - k) \in$ VC, and both directions are reductions.

CLIQUE LEMMA

S is an independent set of $G \Leftrightarrow S$ is a clique of $\bar{G}$ (the complement graph).

So $(G, k) \in$ IS ⇔ $(\bar{G}, k) \in$ CLIQUE. Building $\bar{G}$ costs $O(n^2)$.

Chain: SAT $\leq_p$ 3-SAT $\leq_p$ IS $\leq_p$ VC, CLIQUE. All are NP-complete (each is in NP, via a subset certificate).

Easy special cases exist: VC in **bipartite** graphs is polynomial (König, L12); IS in trees and interval graphs is polynomial (DP, greedy). NP-completeness is about the *general* problem.

Weak vs. strong NP-hardness (the bridge from L07)

- **Subset sum, knapsack, partition:** NP-complete, but solvable in pseudo-polynomial time $O(nT)$. The hardness needs numbers exponential in n: **weakly** NP-hard.
- **TSP, 3-partition, bin packing, graph colouring:** NP-hard even when all numbers are bounded by a polynomial in n: **strongly** NP-hard. No pseudo-polynomial algorithm unless P = NP.

Why it matters for design: for weakly hard problems with *small numbers* in practice, DP is a real solution; for strongly hard ones it is not.

READING A HARDNESS PROOF CRITICALLY

Ask:

1. Which problem was reduced **to** which?
2. Do the constructed instances look like **your** instances (numbers, structure, size)?
3. Does the hardness survive your special structure (planar? bounded degree? small k?)

Hardness of the general problem does not settle your special case.

Exercise B: reductions

1. **3-COLOR ≤ 4-COLOR.** Give the reduction and prove both directions.
2. **HAM-CYCLE ≤ TSP (decision: "a tour of cost $\leq B$?").** Give the construction. Then: what does the construction show about *approximating* TSP when the triangle inequality is not required? (Think about the edge weights you choose.)
3. **Find the flaw.** "To show LONGEST-PATH (is there a simple path with $\geq k$ edges?) is NP-hard, reduce it to 3-SAT by encoding 'vertex v is at position i' with variables $x_{v,i}$."

Groups · 10 min

BLOCK C · 02:10–02:55

Solving hard problems exactly

Backtracking · branch-and-bound · search trees · DP over subsets

Exhaustive search, done intelligently

Backtracking = DFS over partial solutions

```
SEARCH(partial)
  if partial is complete: record it; return
  for each extension e of partial
    if FEASIBLE(partial + e):   ▷ prune infeasible branches
      SEARCH(partial + e)
```

- **Subset sum with pruning** ("stop when the running sum exceeds T") is valid **only for non-negative** numbers. The legacy code did not state this.
- **N-Queens:** one queen per row, columns and diagonals as bit sets, which prunes n^n down to far fewer nodes. The time is still exponential, but the base of the exponent is much smaller.

BRANCH-AND-BOUND (OPTIMISATION)

At each node, compute an **optimistic bound** on any completion. If bound ≤ incumbent (for maximisation), prune.

VALIDITY REQUIREMENT

The bound must be $\geq$ the value of **every** completion of the node (for maximisation). Then pruning never discards an optimum.

LEGACY CORRECTION

The old "B&B knapsack" code pruned only infeasible branches ($W < 0$). That is backtracking with no bound, $O(2^n)$.

Knapsack branch-and-bound with the fractional bound

Sort the items by value/weight. At a node (items $1..i$ decided, weight w, value v):

$$\text{bound} = v + (\text{greedy fractional knapsack on items } i{+}1..n \text{ with capacity } W - w).$$

WHY IT IS VALID

Every 0/1 completion is a feasible solution of the fractional relaxation over the remaining items, and the greedy solves the fractional problem optimally (L08, exercise A1). So bound ≥ every completion's value. ∎

Branch: take item $i+1$ (if it fits) first, then skip it. Prune when bound ≤ incumbent.

WHAT MAKES B&B FAST OR SLOW

- A **tight bound** (close to the true optimum) prunes early.
- A **good incumbent** early (e.g. start from the greedy solution).
- A good **branching order**.

On *strongly correlated* instances ($v_i \approx w_i + c$), all items have similar density, the fractional bound is loose everywhere, and B&B degenerates.

Search trees with provable bounds: Vertex Cover

Question: is there a vertex cover of size $\leq k$?

- **Edge branching:** pick any edge (u, v). Either u or v is in the cover. Branch on both with $k-1$. The tree has $\leq 2^k$ leaves: $O(2^k\, m)$.
- **Degree branching:** pick v of maximum degree d. Either v is in the cover ($k-1$), or **all** of $N(v)$ is ($k-d$). If $d \geq 2$: $T(k) \leq T(k-1) + T(k-2)$, so $O(1.618^k)$. With $d \geq 3$: $O(1.47^k)$. If every degree is ≤ 2, the graph is paths and cycles: solve directly.

These algorithms are exponential **only in** k: they are **fixed-parameter tractable**.

HELD–KARP (1962): DP OVER SUBSETS FOR TSP

$C(S, j)$ = the cheapest path from city 1 that visits exactly S and ends at j:

$$C(S, j) = \min_{i \in S \setminus \{j\}} C(S \setminus \{j\}, i) + d(i, j).$$

$O(n^2 2^n)$ time instead of $(n-1)!$: $n = 25$ is fine, $n = 60$ is not.

Meet-in-the-middle (knapsack/subset sum): $O(2^{n/2} n)$. Inclusion–exclusion: colouring in $O(2^n n)$ (Björklund–Husfeldt–Koivisto 2009).

Experiment: exact methods in practice

0/1 knapsack, W = half the total weight (search nodes; ms)					
n	instances	backtracking nodes	B&B nodes	B&B ms	DP ms
30	uncorrelated	1.24×10^{9}	134	0.0	0.1
100	uncorrelated	–	647	0.0	1.2
30	correlated	1.31×10^{9}	2,512	0.0	0.1
100	**correlated**	**–**	**1.19×10^{9}**	**3,439**	**1.2**

Minimum vertex cover by iterative deepening on k (random graphs, $m = 2n$; total search nodes)			
n	OPT	edge branching	degree branching
30	17	368,612	950
50	26	136,816,087 (3.9 s)	12,139 (2.6 ms)
60	29	–	23,644

`Student_Resources/exact_bench.cpp`, g++ 15.2 -O2.

INTERPRET · 4 MIN

1. B&B vs. DP: which wins, where, and why does DP not care about correlation?

2. Is the $O(nW)$ DP "efficient" here? What would change it?

3. What does the VC table say about worst-case bounds vs. branching rules?

If exact is too expensive: a preview of approximation

MATCHING-VC(G)
C = ∅
while some edge (u,v) has u ∉ C **and** v ∉ C
  C = C ∪ {u, v}
return C

- Polynomial: $O(n + m)$.
- Always returns a vertex cover.
- **Claim:** $|C| \leq 2 \cdot \mathrm{OPT}$, for *every* graph.

Proof: L14. Hint: which lower bound on OPT can you see in the edges the algorithm picked?

RESPONSES TO NP-HARDNESS SO FAR

Exact, exponential	backtracking, B&B, search trees, DP over subsets
Exact, special structure	trees, bipartite graphs, small parameters (FPT)
Exact, pseudo-polynomial	small numbers (L07)
Approximate with a guarantee	**L14**
Heuristics, randomization	L14

Synthesis

1. NP = efficiently **verifiable**. NP-complete = in NP and every NP problem reduces to it.
2. To prove X hard: reduce a known hard problem **to** X. Polynomial construction + (⇒) + (⇐) + membership.
3. The chain SAT → 3-SAT → IS ↔ VC ↔ Clique: gadgets enforce "choose one" and "be consistent".
4. Weakly vs. strongly NP-hard decides whether pseudo-polynomial DP is an option.
5. Exact methods: pruning, **valid** bounds, and smart branching change what is solvable in practice, though not the worst case.

NEXT · L14 (SAT 26 DEC)

Approximation algorithms (ratios, guarantees vs. heuristics; vertex cover, set cover, metric TSP), comparing all responses to hardness, and the course synthesis. Presentations and vivas are also on this day.

Final report due Thu 24 Dec, 23:59. **PS4** due Sat 26 Dec.